

```
import pandas as pd

RAW_DATA_PATH = 'en.openfoodfacts.org.products.csv'
SUBSET_DATA_PATH = 'subset_snacks.csv'

# Only load the columns we actually need for the analysis
COLS_TO_KEEP = [
    'product_name',
    'categories_tags',
    'sugars_100g',
    'proteins_100g',
    'ingredients_text'
]

print("Loading data...")

df = pd.read_csv(
    RAW_DATA_PATH,
    sep='\t',
    usecols=COLS_TO_KEEP,
    nrows=500000,
    low_memory=False,
    on_bad_lines='skip'      # Silently drops any corrupted rows
)

print(f"Successfully loaded {len(df)} rows!")
df.to_csv(SUBSET_DATA_PATH, index=False)
df.head()
```

[Show hidden output](#)

```
print(f"Rows before cleaning: {len(df)}")

# 1. Drop rows missing critical data
df_clean = df.dropna(subset=['product_name', 'sugars_100g', 'proteins_100g']).copy()

# 2. Filter out impossible outliers (Values must be between 0 and 100 per 100g)
df_clean = df_clean[
    (df_clean['sugars_100g'] >= 0) & (df_clean['sugars_100g'] <= 100) &
    (df_clean['proteins_100g'] >= 0) & (df_clean['proteins_100g'] <= 100)
]

print(f"Rows after cleaning: {len(df_clean)}")
print(f"Dropped {len(df) - len(df_clean)} invalid rows.")

# 3. Save this clean baseline
CLEANED_DATA_PATH = 'cleaned_snacks.csv'
df_clean.to_csv(CLEANED_DATA_PATH, index=False)
print(f"Saved clean dataset to {CLEANED_DATA_PATH}")

df_clean.describe()
```

```

Rows before cleaning: 500000
Rows after cleaning: 108087
Dropped 391913 invalid rows.
Saved clean dataset to cleaned_snacks.csv

```

	sugars_100g	proteins_100g
count	108087.000000	108087.000000
mean	12.852077	8.297599
std	18.475758	9.949222
min	0.000000	0.000000
25%	1.009158	2.000000
50%	4.650000	6.000000
75%	16.000000	10.700000
max	100.000000	100.000000

```

# Drop rows that have absolutely no category data
df_categorized = df_clean.dropna(subset=['categories_tags']).copy()
df_categorized.head()

```

	product_name	categories_tags	ingredients_text	sugars_100g	proteins_100g
688	Pinto Bean	en:asian-style-ready-meal	NaN	4.9	17.5
839	pasta	en:beverages-and-beverages-preparations,en:bev...	NaN	1.9	6.7
864	Eirn original curry Sauce	en:plant-based-foods-and-beverages,en:plant-ba...	Wheat Flour Sugar Vegetable Fat D Curry Powder...	29.6	5.2
874	Donut Milka	en:snacks,en:sweet-snacks,en:biscuits-and-cake...	NaN	18.0	6.0
889	Véritable pâte à tartiner noisettes chocolat noir	en:breakfasts,en:spreads,en:sweet-spreads,fr:p...	NaN	32.0	8.0

```

# Define the keyword mapping logic
def map_category(tags):
    # Convert tags to lowercase string to ensure matching works
    tags = str(tags).lower()

    if any(word in tags for word in ['bars', 'energy', 'protein']):
        return 'Energy Bars'
    elif any(word in tags for word in ['chocolate', 'candy', 'sweets', 'confectioneries', 'caramel']):
        return 'Confectionery'
    elif any(word in tags for word in ['cookies', 'biscuits', 'cakes', 'pastries', 'muffins', 'waffles']):
        return 'Baked Goods'
    elif any(word in tags for word in ['chips', 'crisps', 'salty', 'nuts', 'popcorn', 'pretzels']):
        return 'Salty Snacks'
    else:
        return 'Other Snacks'

# Apply the function to create our clean column
df_categorized['primary_category'] = df_categorized['categories_tags'].apply(map_category)

# Check to see how the snacks are distributed
print("Category Distribution:")
print(df_categorized['primary_category'].value_counts())

```

```
# Save the final version for the dashboard
FINAL_DATA_PATH = 'dashboard_data.csv'
df_categorized.to_csv(FINAL_DATA_PATH, index=False)
print(f"\nFinal dashboard data saved to {FINAL_DATA_PATH}")
df_categorized.head()
```

```
Category Distribution:
primary_category
Other Snacks      41784
Confectionery     3212
Salty Snacks      2942
Baked Goods       2937
Energy Bars       1191
Name: count, dtype: int64
```

Final dashboard data saved to dashboard_data.csv

	product_name	categories_tags	ingredients_text	sugars_100g	proteins_100g	primary_category
688	Pinto Bean	en:asian-style-ready-meal	NaN	4.9	17.5	Other Snacks
839	pasta	en:beverages-and-beverages-preparations,en:bev...	NaN	1.9	6.7	Other Snacks
864	Eirn original curry Sauce	en:plant-based-foods-and-beverages,en:plant-ba...	Wheat Flour Sugar Vegetable Fat D Curry Powder...	29.6	5.2	Other Snacks
874	Donut Milka	en:snacks,en:sweet-snacks,en:biscuits-and-	NaN	18.0	6.0	Baked Goods

```
# Define what "High Protein" means (i.e > 15g per 100g)
high_protein_df = df_categorized[df_categorized['proteins_100g'] > 15].copy()

# Drop rows that don't have ingredient text to analyze
high_protein_df = high_protein_df.dropna(subset=['ingredients_text'])
high_protein_df.head()
```

	product_name	categories_tags	ingredients_text	sugars_100g	proteins_100g	primary_category
1004	PRO-TF Chocolate	en:dietary-supplements,en:bodybuilding-supplem...	Whey protein (milk, soy), maltodextrin, egg wh...	5.882353	58.823529	Energy Bars
1470	Formula 1 healthy meal	en:beverages-and-beverages-preparations,en:bev...	Soya protein isolate (40 %), fructose, emulsif...	13.000000	40.000000	Other Snacks
1589	Formula 1 nutritional shake mix fragola delight	en:meals,en:dietary-supplements,en:meal-replac...	Hvetemel, vann, sammalt hvete 17 %, revne hvet...	33.000000	35.000000	Other Snacks

```
# Define protein keywords
protein_keywords = ['whey', 'soy', 'pea', 'peanut', 'casein', 'almond']

# Count occurrences using Pandas vectorized string operations
keyword_counts = {}
for keyword in protein_keywords:
    count = high_protein_df['ingredients_text'].str.lower().str.contains(keyword).sum()
    keyword_counts[keyword] = count

# Convert to a Series, sort it
top_proteins = pd.Series(keyword_counts).sort_values(ascending=False)
```

```
print(f"Analyzed {len(high_protein_df)} high-protein products.")  
print("\nTop Protein Sources:")  
print(top_proteins)
```

Analyzed 4651 high-protein products.

Top Protein Sources:

soy	928
pea	755
peanut	588
almond	304
whey	255
casein	54
dtype:	int64